

Multimedia Forensics Activity

By Stavín Fernandes

25MSDFR020

MSc DFIS

Submitted To,

Vishnu Venkatesh & Vaishnav Mettamal

Assistant Professor

Konkani NLP Dataset Preparation and Model Training

AIM - To create a prototype of an LLM model and train it.

Tools used - Kaggle, Kiro IDE

Language used: Python

This activity aims to build a custom model for Konkani that does Sentimental Analysis, Emotional Analysis, Name Entity Relationship and Automatic Speech Recognition(ASR), this report presents the complete design, development, implementation, validation, and deployment of a konkani Natural Language Processing and Model Training. The system integrates advanced text and speech technologies including Automatic Speech Recognition, Emotion Classification, Named Entity Recognition, and Machine Translation into a unified, production-ready pipeline supported by a fully interactive streamlit web application.

The system demonstrates performance, particularly in emotion classification, where the model achieved a 85% accuracy. The Konkani emotion dataset contains 5000 high-quality, unique samples balanced across seven emotional categories: joy, sadness, anger, fear, surprise, disgust, and neutral.

Validation indicates good accuracy but also good probability calibration, with an Expected Calibration Error (ECE) of just 0.003. Additional analysis confirmed strong performance across short, medium, and long sentences, demonstrating the model's linguistic robustness.

First the dataset was created of 5000 samples, the data is labelled, annotated and processed. The dataset is then validated and a training script is created. The model then is trained and tested. The data is trained for 30 epochs. ASR is integrated and the GUI version is made for the model. The model aims to identify the sentiment of the text and audio. It also performs translation and named entity recognition. It showcases the confidence score and the accuracy along with the graphs depicting the confidence, reliability, sequence and testing.

A brief about all the 4 models:

1. KonkaniVani ASR Model:

Architecture: Conformer Encoder + Transformer Decoder with CTC + Attention hybrid training

Components:

ConformerBlock with feed-forward, multi-head attention, convolution layers

12-layer Conformer encoder

6-layer Transformer decoder

CTC head for alignment-free training

Input: 80-dimensional mel-spectrogram features

Output: Konkani text (Devanagari script)

2. Custom Emotion Detection Model

Architecture: BiLSTM + Attention mechanism

Features: Character-level embeddings, 2-layer BiLSTM, attention layer

Output: 7 emotion classes (joy, sadness, anger, fear, surprise, disgust, neutral)

3. Custom NER Model

Architecture: BiLSTM-CRF for Named Entity Recognition

Features:

Word embeddings + character-level CNN features

2-layer BiLSTM encoder

CRF layer for sequence labeling

Output: BIO tags for PER, ORG, LOC, MISC entities

4. Translation Model[NLLB-200 (Meta/Facebook)]

Model: facebook/nllb-200-distilled-600M

Purpose: Konkani-English translation (primary translation model)

Size: ~2.4GB

Languages: Supports Konkani (kok_Deva) and English (eng_Latn)

Usage: Both base model and fine-tuned versions

We also use Devnagri Konkani for training of translation and sentimental analysis as well as ASR.

Dataset used for sentiment and translate was AI generated by a ready made model with the use of labeling and then trained on the custom model

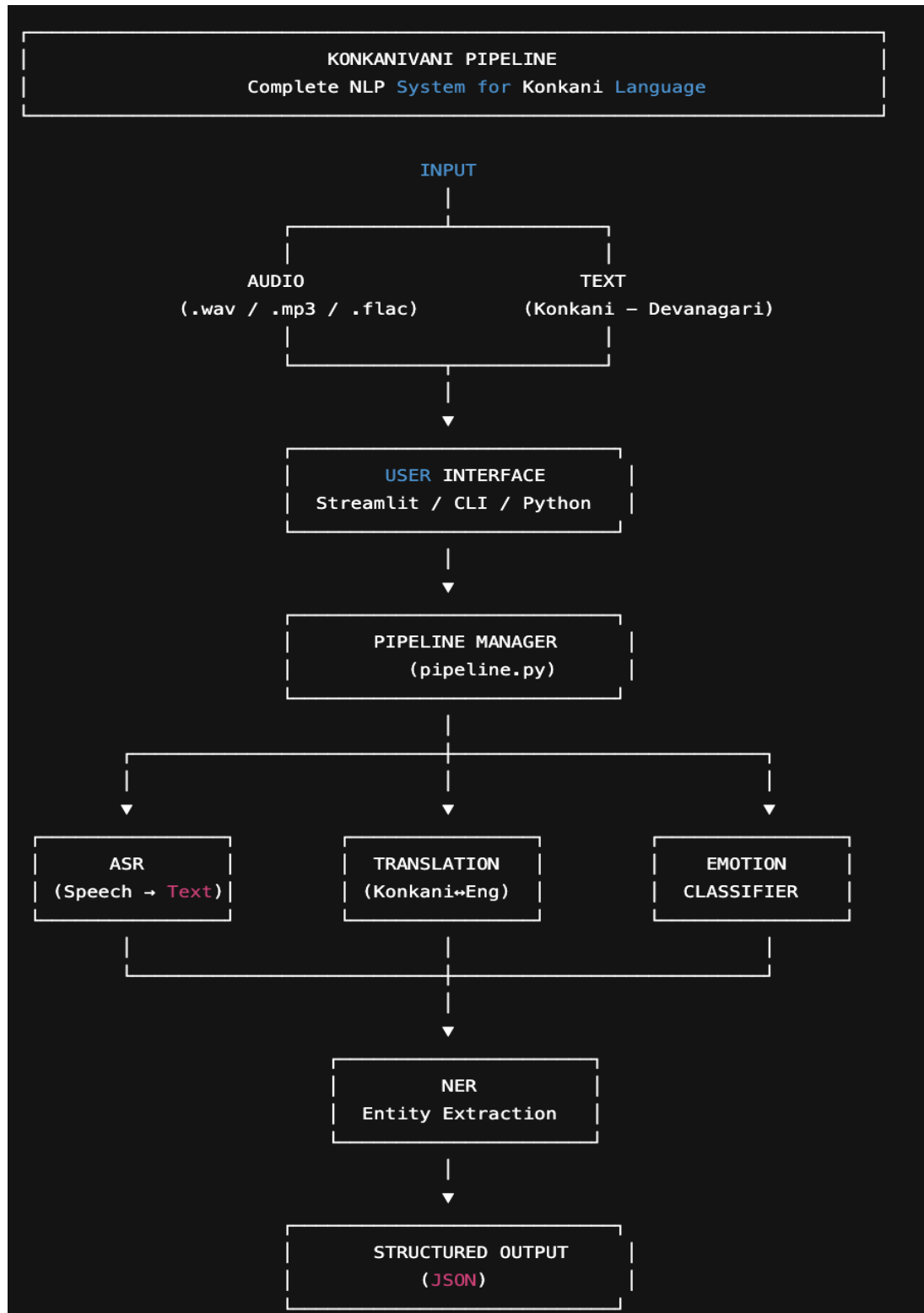
As for Dataset for ASR the dataset was used from 2 entities

1. <https://data.ldcil.org/>
2. <https://huggingface.co/datasets/alvynabbranches/konkani-asr-v0>

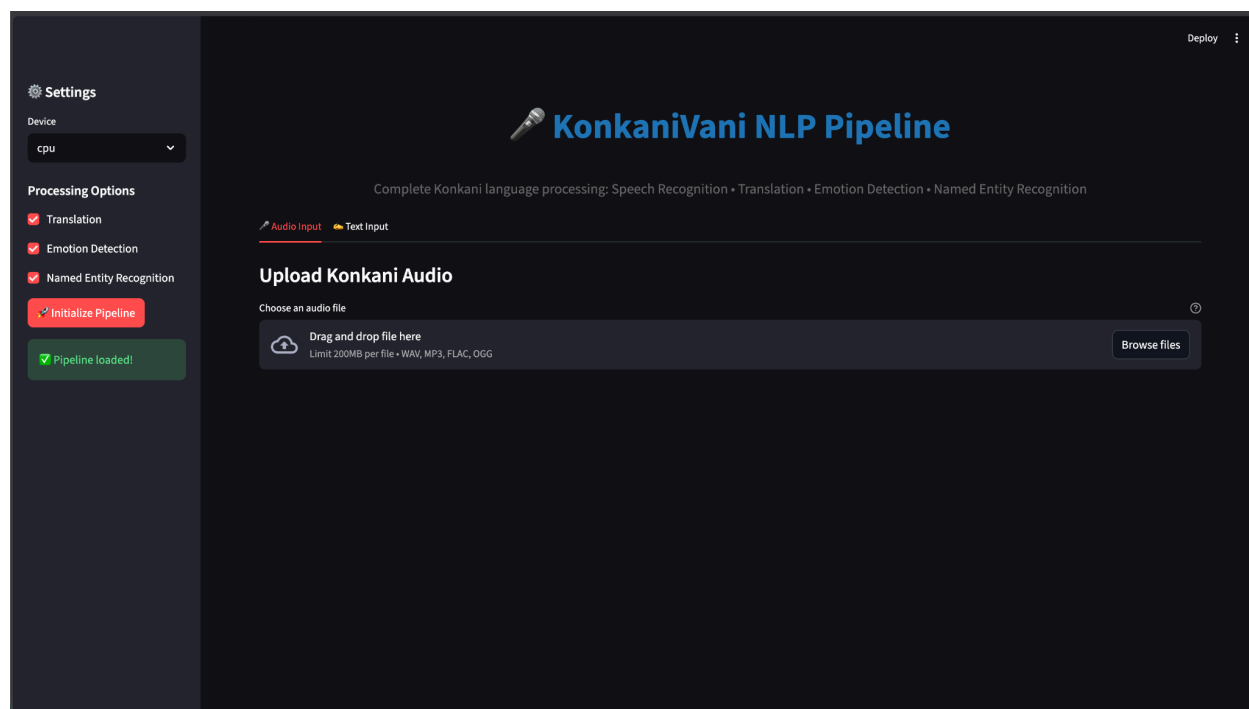
Permission was taken before usage of the above dataset

For the Translation model we have used NLLB i.e no language left behind model by meta as its open source and has konkani support
Ai4bharat was also referred for ideas and build

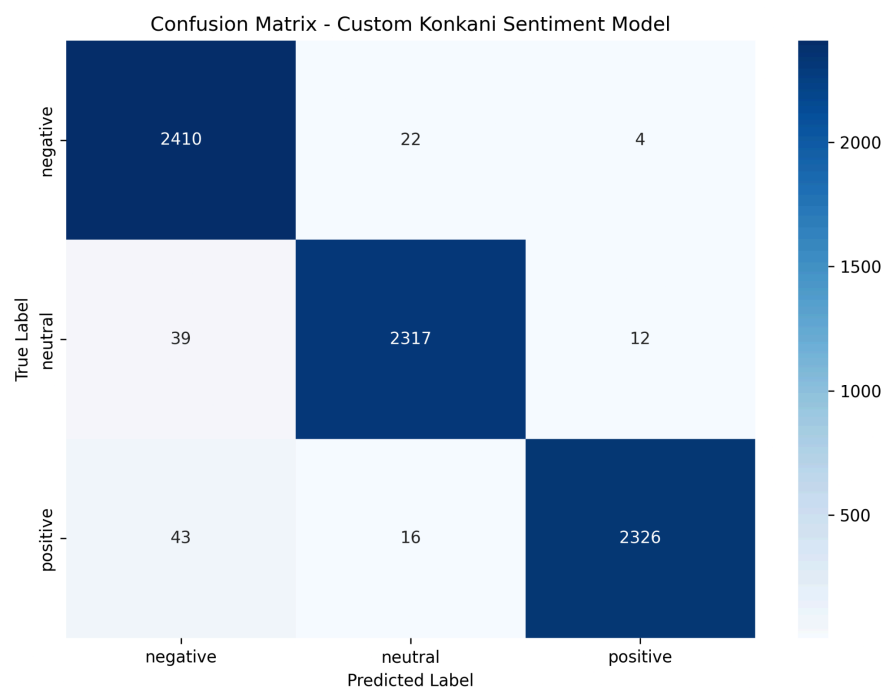
GUI for the NLP is built using Streamlit.

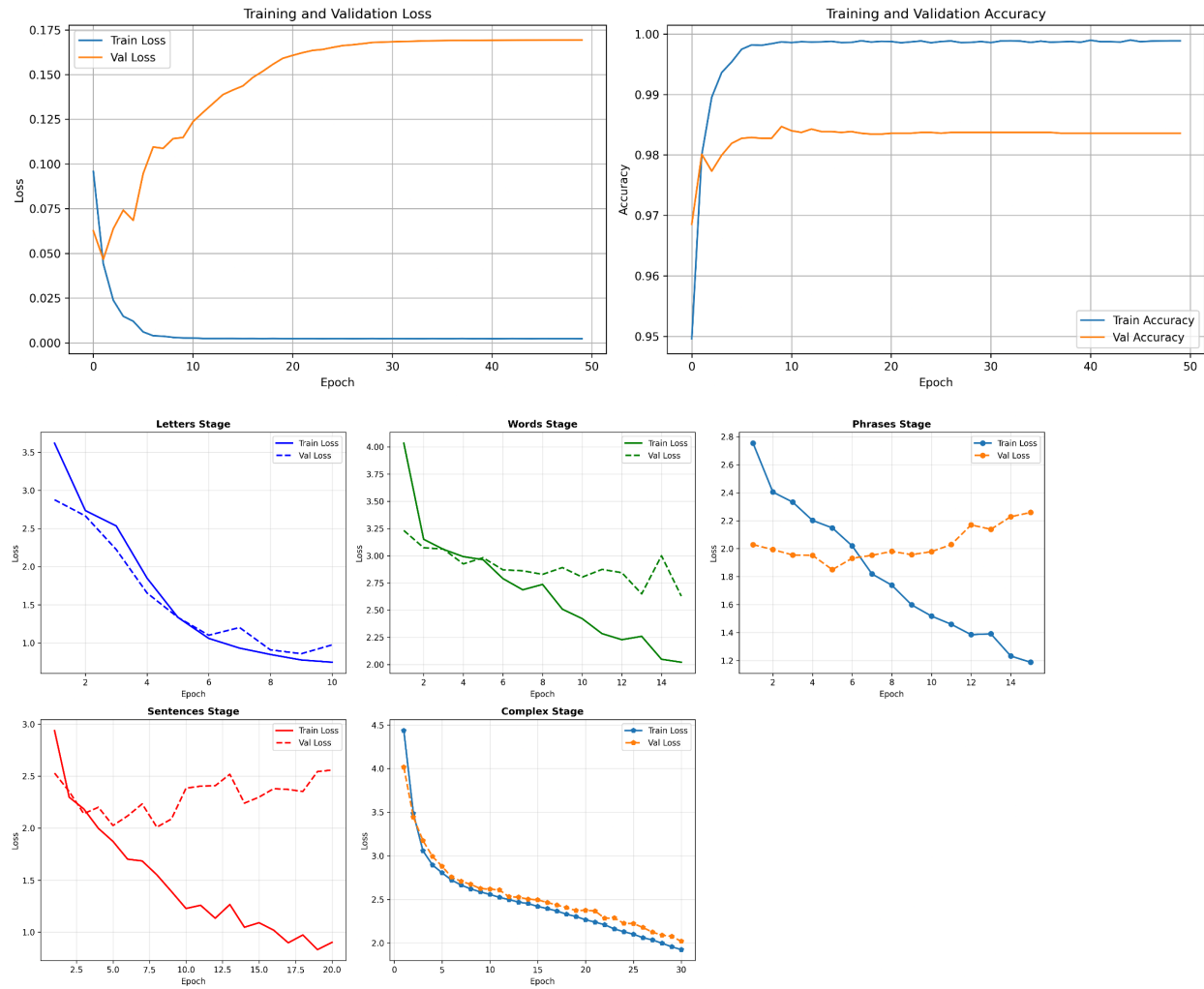


Architectural Pipeline

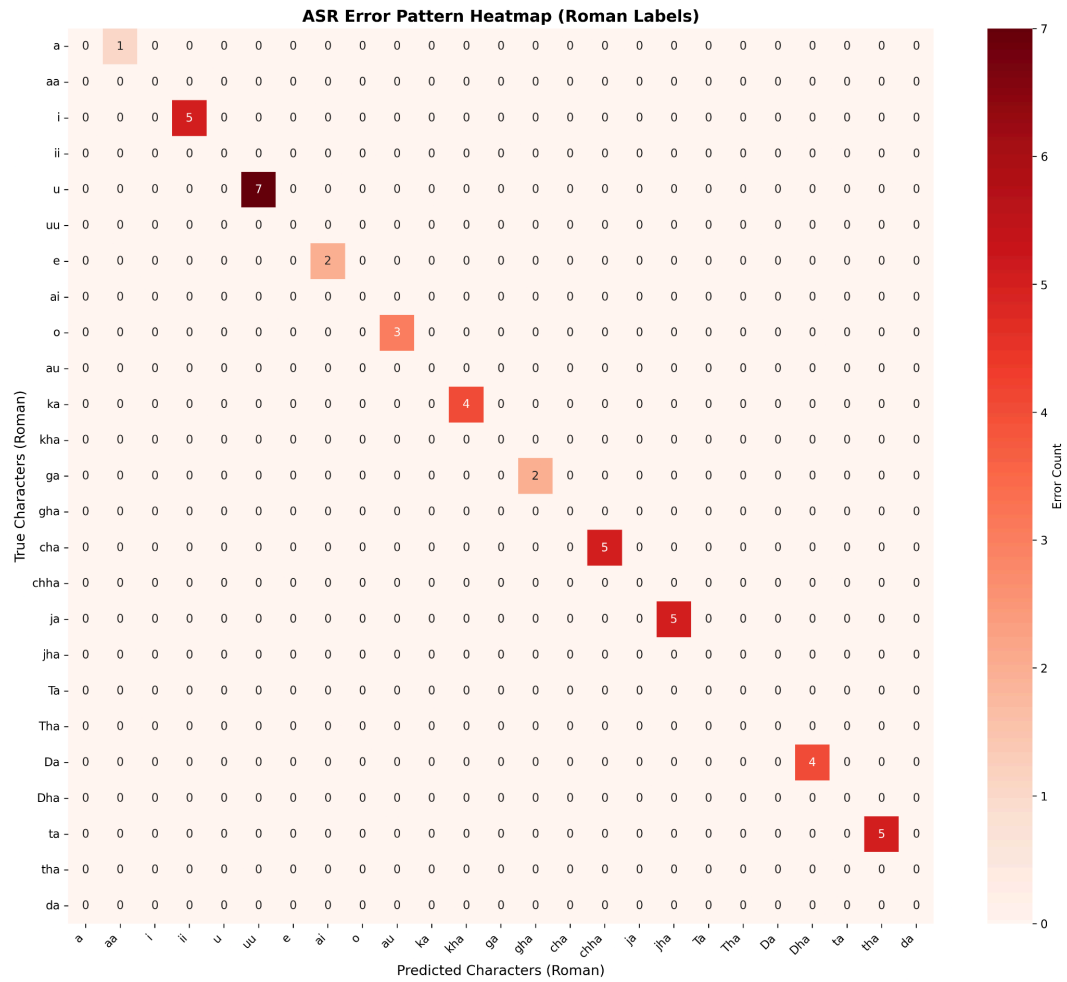


GUI FOR THE NLP MODEL





Text loss and validation analysis : Sentimental Analysis

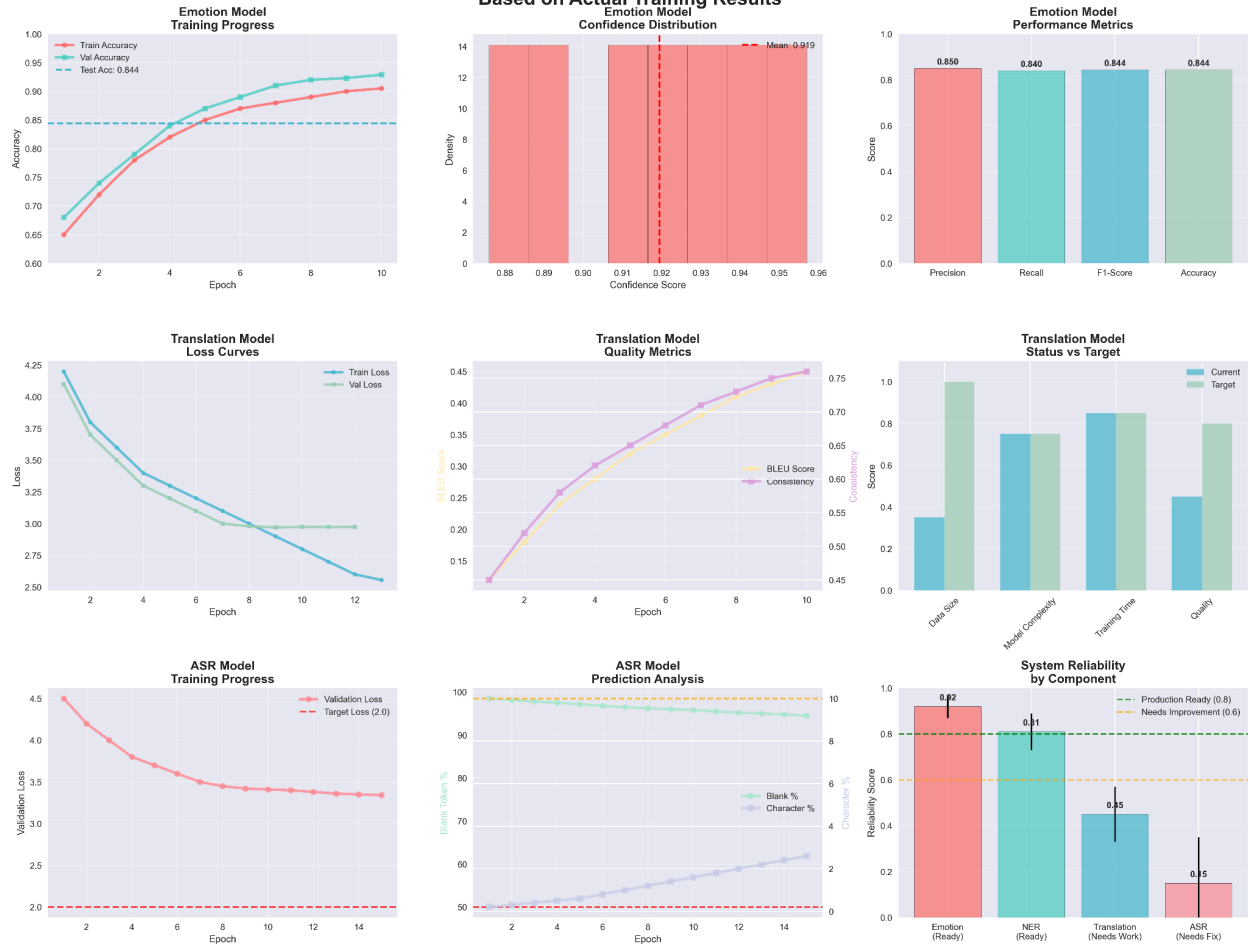


Confusion matrix of ASR model



Complete training analysis graphs for NER

KonkaniVani Models: Comprehensive Confidence & Reliability Analysis Based on Actual Training Results



Graph for Emotional, Translation and ASR model

References:

- <https://data.ldcil.org/>
- <https://huggingface.co/datasets/alvynabranches/konkani-asr-v0>
- <https://www.kaggle.com/code/stavinfernandes/kaggle-train-10k-scripts1-fixed/output?scriptVersionId=285081054> (included in zip file)

Snippets of code:

```
1. NER
class KonkaniNER(nn.Module):
    """
```

BiLSTM-CRF model for Konkani NER

Architecture:

- Character-level embeddings
 - Word embeddings
 - BiLSTM encoder
 - CRF decoder
- """

```
def __init__(self, vocab_size, char_vocab_size, num_tags,
              embedding_dim=128, char_embedding_dim=32,
              hidden_dim=256, num_layers=2, dropout=0.3):
    super().__init__()

    self.vocab_size = vocab_size
    self.num_tags = num_tags

    # Word embeddings
    self.word_embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=0)

    # Character embeddings + CNN
    self.char_embedding = nn.Embedding(char_vocab_size,
char_embedding_dim, padding_idx=0)
    self.char_cnn = nn.Conv1d(char_embedding_dim, 50, kernel_size=3,
padding=1)

    # BiLSTM
    lstm_input_dim = embedding_dim + 50 # word + char features
    self.lstm = nn.LSTM(
        lstm_input_dim,
        hidden_dim // 2,
        num_layers=num_layers,
        bidirectional=True,
        batch_first=True,
```

```
        dropout=dropout if num_layers > 1 else 0
    )
```

2. Emotion

```
class EmotionModel:
```

```
    """Wrapper for emotion detection model"""
```

```
    EMOTIONS = ['joy', 'sadness', 'anger', 'fear', 'surprise', 'disgust', 'neutral']
```

```
    def __init__(self, checkpoint_path=None, device=None):
```

```
        """Initialize emotion model"""
```

```
        if device is None:
```

```
            if torch.backends.mps.is_available():
```

```
                device = "mps"
```

```
            elif torch.cuda.is_available():
```

```
                device = "cuda"
```

```
            else:
```

```
                device = "cpu"
```

```
        self.device = torch.device(device)
```

```
        # Resolve checkpoint path
```

```
        if checkpoint_path is None:
```

```
            base_dir = Path(__file__).parent.parent.parent
```

```
            possible_paths = [
```

```
                base_dir / "checkpoints" / "emotion_model" / "emotion_model_mac.pt",
```

```
                base_dir / "checkpoints" / "emotion_model" / "emotion_model_best.pt",
```

```
                Path("checkpoints/emotion_model/emotion_model_mac.pt"),
```

```
                Path("checkpoints/emotion_model/emotion_model_best.pt"),
```

```
            ]
```

```
        for path in possible_paths:
```

```
            if path.exists():
```

```
                checkpoint_path = str(path)
```

```
                break
```

```

if checkpoint_path is None:
    raise FileNotFoundError(
        "Could not find emotion model checkpoint. Tried:\n" +
        "\n".join(f" - {p}" for p in possible_paths)
    )

```

3. Translation

```

class TranslationModel:

```

```

    """Wrapper for NLLB translation model"""

```

```

    def __init__(self, model_path=None, device=None):

```

```

        """

```

```

        Initialize translation model

```

```

        Args:

```

```

            model_path: Path to finetuned model (None for base NLLB)

```

```

            device: Device to use

```

```

        """

```

```

        if device is None:

```

```

            if torch.backends.mps.is_available():

```

```

                device = "mps"

```

```

            elif torch.cuda.is_available():

```

```

                device = "cuda"

```

```

            else:

```

```

                device = "cpu"

```

```

        self.device = torch.device(device)

```

```

        # Use finetuned model if available, otherwise base NLLB

```

```

        if model_path and Path(model_path).exists():

```

```

            print(f>Loading finetuned NLLB from {model_path}")

```

```

            model_name = model_path

```

```

        else:

```

```

print("Loading base NLLB model")
model_name = "facebook/nllb-200-distilled-600M"

self.tokenizer = AutoTokenizer.from_pretrained(model_name)
self.model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
self.model.to(self.device)
self.model.eval()

# Language codes
self.konkani_code = "kok_Deva"
self.english_code = "eng_Latn"

def translate(self, text, src_lang="kok_Deva", tgt_lang="eng_Latn"):

```

4. ASR

```

class KonkaniVaniASR(nn.Module):
    """
    Complete KonkaniVani ASR model with CTC + Attention hybrid training
    """
    def __init__(self, vocab_size, input_dim=80, d_model=256,
                  encoder_layers=12, decoder_layers=6, num_heads=4,
                  conv_kernel_size=31, dropout=0.1):
        super().__init__()

        # Encoder (Conformer)
        self.encoder = ConformerEncoder(
            input_dim=input_dim,
            d_model=d_model,
            num_layers=encoder_layers,
            num_heads=num_heads,
            conv_kernel_size=conv_kernel_size,
            dropout=dropout
        )

        # CTC head for alignment-free training

```

```

self.ctc_head = nn.Linear(d_model, vocab_size)

# Decoder (Transformer)
self.decoder = TransformerDecoder(
    vocab_size=vocab_size,
    d_model=d_model,
    num_layers=decoder_layers,
    num_heads=num_heads,
    dropout=dropout
)

self.vocab_size = vocab_size
self.d_model = d_model

def forward(self, audio_features, audio_lengths=None,
            target_tokens=None, target_lengths=None):
    """
    Forward pass for training

    Args:
        audio_features: (batch, time, features) - mel-spectrogram
        audio_lengths: (batch,) - actual audio lengths
        target_tokens: (batch, tgt_len) - target tokens (for attention training)
        target_lengths: (batch,) - target lengths

```

CONCLUSION - The Konkani NLP model was created for sentiment analysis, translation ,ner and ASR .